

# **Project Title**

**QA Management Suite: A Web-Based Software Testing and Defect Management System**

# 1. Introduction

Software Requirement Specification (SRS) is a document that describes the functional and non-functional requirements of the QA Management Suite. It defines the expected behavior, features, limitations, and requirements of the proposed system.

The QA Management Suite is a web-based software testing and defect management platform designed to support software quality assurance activities. The system provides a centralized environment where QA engineers, developers, and administrators can manage projects, requirements, test cases, test executions, and software defects.

This document provides a detailed description of the system requirements that will guide the design, development, and testing phases of the project.

## 2. Purpose

The main purpose of this system is to provide an organized platform for managing software testing activities throughout the Software Development Life Cycle (SDLC).

The system aims to:

- Reduce dependency on manual testing management methods.
- Improve communication between QA engineers and developers.
- Provide efficient defect tracking and resolution.
- Maintain proper records of testing activities.
- Improve visibility of software quality through reports and dashboards.

## 3. Scope

The QA Management Suite will provide features required to manage software quality assurance activities within a software development environment.

The system will support:

- User authentication and authorization.
- Project management.
- Requirement management.
- Test case creation and execution.
- Defect reporting and tracking.
- Developer assignment and defect resolution.
- Testing reports and dashboards.

The system will mainly target small software development teams, QA teams, and educational software projects.

## 4. Overall Description

The QA Management Suite is designed as a web-based application using a client-server architecture. Users interact with the system through a web interface, while backend services process requests and communicate with the database.

The system consists of three main components:

Frontend Application

Responsible for providing user interfaces and user interactions.

Backend Application

Responsible for business logic, authentication, API handling, and data processing.

Database

Responsible for storing user information, project details, test cases, and defect records.

## 5. User Roles

The QA Management Suite provides different functionalities based on user roles. Each user type has specific responsibilities and permissions within the system.

---

### 5.1 Administrator

The Administrator manages the overall system operations.

Responsibilities:

- Manage user accounts.
  - Assign user roles and permissions.
  - Create and manage projects.
  - Monitor system activities.
  - View overall reports and statistics.
  - Maintain system settings.
- 

### 5.2 QA Engineer

The QA Engineer is responsible for performing software testing activities.

Responsibilities:

- View assigned projects.
  - Manage requirements.
  - Create and maintain test cases.
  - Execute test cases.
  - Record test results.
  - Report software defects.
  - Perform defect retesting.
  - Generate testing reports.
- 

### 5.3 Developer

The Developer is responsible for analyzing and resolving reported defects.

Responsibilities:

- View assigned defects.
  - Analyze bug details.
  - Update defect status.
  - Add comments regarding fixes.
  - Provide resolution information.
  - Collaborate with QA engineers.
- 

## 6. Functional Requirements

Functional requirements define the operations and services that the QA Management Suite must provide.

---

### 6.1 User Authentication and Authorization

Description:

The system shall provide secure user authentication and access control.

Requirements:

**FR-001:**

The system shall allow registered users to log in using email and password.

**FR-002:**

The system shall validate user credentials before granting access.

**FR-003:**

The system shall provide different access permissions based on user roles.

**FR-004:**

The system shall allow users to log out securely.

**FR-005:**

The system shall prevent unauthorized users from accessing restricted features.

---

### 6.2 User Management

Description:

The system shall allow administrators to manage system users.

Requirements:

**FR-006:**

The administrator shall be able to create user accounts.

**FR-007:**

The administrator shall be able to update user information.

**FR-008:**

The administrator shall assign roles to users.

**FR-009:**

The administrator shall activate or deactivate user accounts.

---

## 6.3 Project Management

Description:

The system shall provide facilities to manage software projects.

Requirements:

**FR-010:**

Users shall be able to create projects.

**FR-011:**

Users shall be able to view project details.

**FR-012:**

The system shall allow members to be assigned to projects.

**FR-013:**

The system shall maintain project status information.

---

## 6.4 Requirement Management

Description:

The system shall support requirement tracking.

Requirements:

**FR-014:**

QA engineers shall be able to add project requirements.

**FR-015:**

Users shall be able to update requirement information.

**FR-016:**

The system shall link requirements with related test cases.

**FR-017:**

The system shall maintain requirement history.

## **6.5 Test Case Management**

Description:

The system shall provide functionalities to create, manage, and organize software test cases.

Requirements:

**FR-018:**

The QA Engineer shall be able to create new test cases.

**FR-019:**

The system shall allow users to add test case details including title, description, test steps, and expected results.

**FR-020:**

The system shall allow users to edit existing test cases.

**FR-021:**

The system shall allow users to delete test cases when required.

**FR-022:**

The system shall categorize test cases based on project modules.

**FR-023:**

The system shall maintain test case history and updates.

**FR-024:**

The system shall allow users to search and filter test cases.

---

## 6.6 Test Execution Management

Description:

The system shall allow QA engineers to execute test cases and record testing results.

Requirements:

**FR-025:**

QA engineers shall be able to select test cases for execution.

**FR-026:**

The system shall allow users to mark test results as:

- Pass
- Fail



- Blocked

**FR-027:**

The system shall store test execution details.

**FR-028:**

The system shall allow users to add execution comments.

**FR-029:**

The system shall maintain execution history.

**FR-030:**

The system shall generate test execution summaries.

---

## 6.7 Defect/Bug Management

Description:

The system shall provide a complete defect tracking process from reporting to resolution.

Requirements:

**FR-031:**

QA engineers shall be able to create defect reports.

**FR-032:**

The system shall store defect information including:

- Bug title
- Description
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Status

**FR-033:**

The system shall allow screenshots or files to be attached to defects.

**FR-034:**

The system shall allow defects to be assigned to developers.

**FR-035:**

Developers shall be able to update defect status.

**FR-036:**

The system shall support defect lifecycle management:

- New
- Assigned
- In Progress
- Fixed
- Retesting
- Closed
- Reopened

**FR-037:**

Users shall be able to add comments to defect records.

**FR-038:**

The system shall maintain defect history.

---

## 6.8 Dashboard and Reporting

Description:

The system shall provide dashboards and reports to monitor software quality.

Requirements:

**FR-039:**

The system shall display project testing statistics.

**FR-040:**

The dashboard shall display:

- Total projects
- Total test cases
- Passed tests
- Failed tests
- Open defects
- Closed defects

**FR-041:**

The system shall provide graphical charts for testing progress.

**FR-042:**

Users shall be able to generate testing reports.

**FR-043:**

The system shall provide defect summary reports.

## **6.9 Notification Management**

Description:

The system shall provide notifications to inform users about important activities and updates.

Requirements:

**FR-044:**

The system shall notify users when a defect is assigned to them.

**FR-045:**

The system shall notify QA engineers when developers update defect status.

**FR-046:**

The system shall notify users about test case updates.

**FR-047:**

The system shall maintain notification history.

---

## 6.10 Search and Filtering

Description:

The system shall provide searching and filtering capabilities to improve usability.

Requirements:

**FR-048:**

Users shall be able to search projects, test cases, and defects.

**FR-049:**

The system shall allow filtering defects based on:

- Status
- Priority
- Severity
- Assigned developer

**FR-050:**

The system shall allow filtering test cases based on project and module.

---

## 6.11 File Attachment Management

Description:

The system shall support uploading and managing files related to testing activities.

Requirements:

**FR-051:**

Users shall be able to attach screenshots to defect reports.

**FR-052:**

The system shall store attachment information securely.

**FR-053:**

Users shall be able to view attached files.

---

## 7. Non-Functional Requirements

Non-functional requirements define the quality attributes and performance expectations of the QA Management Suite.

---

### 7.1 Performance Requirements

**NFR-001:**

The system should provide fast response time for user operations.

**NFR-002:**

The system should handle multiple users accessing the application simultaneously.

**NFR-003:**

Database queries should be optimized to improve system performance.

---

### 7.2 Security Requirements

**NFR-004:**

The system shall provide secure user authentication.

**NFR-005:**

User passwords shall be encrypted before storing in the database.

**NFR-006:**

The system shall implement role-based access control.

**NFR-007:**

Unauthorized users shall not access restricted information.

**NFR-008:**

User sessions shall be managed securely.

---

## **7.3 Usability Requirements**

**NFR-009:**

The system interface should be simple and user-friendly.

**NFR-010:**

Users should be able to learn the system features easily.

**NFR-011:**

The system should provide meaningful error messages.

**NFR-012:**

The interface should support responsive design for different screen sizes.

---

## **7.4 Reliability Requirements**

**NFR-013:**

The system shall maintain accurate records of testing activities.

**NFR-014:**

The system should prevent data loss during normal operations.

**NFR-015:**

The system should recover from unexpected failures.

## **7.5 Maintainability Requirements**

**NFR-016:**

The system should follow a modular software architecture to simplify future modifications.

**NFR-017:**

The source code should be properly structured and documented.

**NFR-018:**

The system should support easy identification and fixing of software issues.

**NFR-019:**

New features should be added without affecting existing functionalities.

---

## 7.6 Scalability Requirements

**NFR-020:**

The system should support adding new users, projects, and testing records.

**NFR-021:**

The database structure should support future expansion.

**NFR-022:**

The system should allow additional modules to be integrated in future versions.

---

## 7.7 Compatibility Requirements

**NFR-023:**

The system should operate on modern web browsers.

Supported browsers:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

**NFR-024:**

The system should support different operating systems including:

- Windows
- macOS
- Linux

---

## 8. System Features

The QA Management Suite consists of several main features that support the complete software testing lifecycle.

---

### 8.1 Authentication Module

Description:

Provides secure access to the system.

Features:

- User login
  - User logout
  - Role-based access control
  - Password security
- 

### 8.2 Project Management Module

Description:

Allows users to manage software projects.

Features:

- Create projects
  - Update project details
  - Add team members
  - View project status
- 

### 8.3 Requirement Management Module

Description:

Allows QA teams to manage software requirements.



Features:

- Add requirements
  - Update requirements
  - Track requirement status
  - Link requirements with test cases
- 

## 8.4 Test Case Management Module

Description:

Allows QA engineers to create and maintain testing scenarios.

Features:

- Create test cases
  - Define testing steps
  - Add expected results
  - Update test cases
  - Search test cases
- 

## 8.5 Test Execution Module

Description:

Supports execution and monitoring of test activities.

Features:

- Execute test cases
  - Record results
  - Add execution notes
  - View execution history
- 

## 8.6 Defect Management Module

Description:

Manages the complete bug lifecycle.

Features:

- Report defects
- Assign defects
- Update defect status
- Add comments
- Upload screenshots
- Track defect history

## **8.7 Dashboard and Reporting Module**

Description:

The dashboard provides a visual overview of project quality status and testing progress.

Features:

- Display project testing statistics.
  - Display defect statistics.
  - Show test execution progress.
  - Provide graphical charts.
  - Generate testing summary reports.
  - Monitor overall software quality.
- 

## 8.8 Notification Module

Description:

The notification module keeps users informed about important system activities.

Features:

- Notify developers when bugs are assigned.
  - Notify QA engineers about bug status changes.
  - Display system alerts.
  - Maintain notification history.
- 

## 8.9 User Management Module

Description:

Allows administrators to control system users and permissions.

Features:

- Create user accounts.
  - Update user details.
  - Assign roles.
  - Manage user access permissions.
  - Activate or deactivate accounts.
-

## 9. External Interface Requirements

External interface requirements define how users and external components interact with the QA Management Suite.

---

### 9.1 User Interface Requirements

The system shall provide a web-based user interface.

The user interface should include:

- Login page.
- Dashboard.
- Navigation menu.
- Project management pages.
- Test case management pages.
- Bug management pages.
- Report pages.

The interface should be:

- Simple.
  - Responsive.
  - User-friendly.
  - Easy to navigate.
- 

### 9.2 Software Interface Requirements

The system will communicate with the following software components:

Frontend:

- React.js application

Backend:

- Node.js and Express.js API services

Database:

- MongoDB database

Testing Tools:

- Postman
  - Selenium WebDriver
- 

## 9.3 Hardware Interface Requirements

The system does not require special hardware.

Minimum requirements:

### Client Device:

- Computer or laptop
- Internet connection
- Modern web browser

### Server Environment:

- Application server
  - Database server
- 

## 9.4 API Interface Requirements

The system will use RESTful APIs for communication between frontend and backend.

Examples:

### Authentication API

POST /api/auth/login

### Project API

GET /api/projects

POST /api/projects

### Test Case API

GET /api/testcases

POST /api/testcases

### Bug API

GET /api/bugs

POST /api/bugs

PUT /api/bugs/:id

---

## 10. Database Requirements

The system database shall store and manage all application data required for system operations.

### **Main Database Collections/Tables:**

#### Users

Stores user account information.

Attributes:

- user\_id
  - name
  - email
  - password
  - role
  - created\_date
- 

#### Projects

Stores project information.

Attributes:

- project\_id
  - project\_name
  - description
  - status
  - created\_date
- 

#### Requirements

Stores software requirements.

Attributes:

- requirement\_id
- project\_id
- title
- description
- priority

- status
- 

## Test Cases

Stores testing scenarios.

Attributes:

- test\_case\_id
- project\_id
- title
- steps
- expected\_result
- status

## 10. Database Requirements

The database of the QA Management Suite will maintain all information required for managing software testing activities. The database should ensure data consistency, security, and efficient retrieval of information.

---

## Test Execution Collection

Stores test execution results.

Attributes:

- execution\_id
  - test\_case\_id
  - executed\_by
  - result
  - execution\_date
  - comments
- 

## Defect/Bug Collection

Stores reported software defects.

Attributes:

- bug\_id
  - project\_id
  - title
  - description
  - steps\_to\_reproduce
  - expected\_result
  - actual\_result
  - severity
  - priority
  - status
  - assigned\_to
  - created\_by
  - created\_date
- 

## Comment Collection

Stores comments related to defects.

Attributes:

- comment\_id
- bug\_id
- user\_id
- comment\_text



- created\_date
- 

## Attachment Collection

Stores files attached to defects.

Attributes:

- attachment\_id
  - bug\_id
  - file\_name
  - file\_path
  - uploaded\_by
  - uploaded\_date
- 

## Notification Collection

Stores system notifications.

Attributes:

- notification\_id
  - user\_id
  - message
  - status
  - created\_date
- 

## Database Relationships

The main relationships between database entities are:

- One user can create multiple projects.
- One project can contain multiple requirements.
- One requirement can contain multiple test cases.
- One test case can have multiple execution records.
- One project can contain multiple defects.
- One defect can have multiple comments and attachments.
- One user can receive multiple notifications.

---

## 11. System Constraints

The following constraints define limitations and conditions of the system.

### Technical Constraints

- The system will be developed as a web-based application.
- The system requires an internet connection for online access.
- The application will depend on the selected technology stack.
- The system performance depends on server and database resources.

### Security Constraints

- Users must authenticate before accessing system features.
- Access permissions depend on user roles.
- Sensitive user information must be protected.

### Development Constraints

- The project will be developed within the allocated academic project timeline.
- Development will follow available resources and technical knowledge.
- Third-party integrations will be limited to required services only.

---

## 12. Assumptions and Dependencies

### Assumptions

- Users have basic knowledge of software testing activities.
- Users have access to a modern web browser.
- Required project information will be provided by authorized users.
- Users will maintain accurate testing records.

### Dependencies

The system depends on:

- React.js framework for frontend development.
- Node.js and Express.js for backend services.
- MongoDB for database management.
- Internet connectivity for deployment and access.

- Third-party libraries required for development.

## **13. Future Enhancements**

The current version of the QA Management Suite focuses on essential software testing and defect management activities. However, several advanced features can be introduced in future versions to improve system capabilities and efficiency.

### **13.1 AI-Based Defect Analysis**

The system can be enhanced with Artificial Intelligence features to analyze historical defects and suggest:

- Bug severity prediction.
  - Priority recommendations.
  - Possible duplicate defect detection.
  - Common defect patterns.
- 

### 13.2 AI-Assisted Test Case Generation

An AI-based module can be introduced to automatically generate test cases based on software requirements.

Benefits:

- Reduces manual effort.
  - Improves test coverage.
  - Helps QA engineers identify missing scenarios.
- 

### 13.3 CI/CD Pipeline Integration

The system can be integrated with Continuous Integration and Continuous Deployment tools.

Possible integrations:

- GitHub Actions.
- Jenkins.
- GitLab CI/CD.

Benefits:

- Automatic test execution.
  - Automated test result collection.
  - Faster defect identification.
- 

### 13.4 Mobile Application Support

A mobile application version can be developed to allow users to:

- View assigned tasks.
  - Update defect status.
  - Monitor testing progress remotely.
-

### 13.5 Advanced Analytics

Future versions can include advanced analytics features such as:

- Quality trend analysis.
  - Defect prediction reports.
  - Team performance analysis.
  - Release quality evaluation.
- 

### 13.6 Integration with External Tools

The system can be extended by integrating with existing software development tools such as:

- Jira.
- GitHub.
- GitLab.
- Slack.

This will improve collaboration between development and testing teams.

---

## 14. Conclusion

The Software Requirement Specification document defines the requirements and expected functionality of the QA Management Suite. The proposed system provides a centralized platform to manage software testing activities, including requirement management, test case management, test execution, defect tracking, and quality reporting.

The system aims to overcome limitations of traditional manual testing management approaches by providing an organized and efficient solution for QA teams. Through role-based access, dashboards, reporting features, and defect lifecycle management, the application improves communication between QA engineers and developers while increasing software quality management efficiency.

The development of this system will demonstrate practical knowledge in software engineering, full-stack web development, database management, and software quality assurance practices. The proposed QA Management Suite can serve as a valuable solution for small development teams and educational environments while providing opportunities for future expansion with advanced technologies.

